Tomasulo Algorithm: Multi-Issue Simulation

Ahmed Ahmed
Computer Architecture | October 15, 2026

1 Introduction

For this assignment, I worked with the supplied Python Tomasulo simulator and extended it to issue up to four instructions per cycle. I wanted to see how much a wider issue stage would improve execution time while keeping the final register and memory values correct. I tested issue widths of 1, 2, 3, and 4, using the same hardware settings within each test case.

I kept instruction issue and commit in program order. Ready instructions can execute out of order, which allows the integer and floating-point units to work on different instructions at the same time. My implementation uses one common data bus and one commit slot per cycle.

What works

My simulator supports register renaming, configurable reservation stations and functional units, branch prediction and recovery, store-to-load forwarding, and memory replay. I can also print the cycle for each pipeline stage and inspect the final registers and nonzero memory values.

Limitations

I used one non-pipelined memory port and modeled memory as 64 aligned words at byte addresses 0-252. Floating-point operations use Python arithmetic, so the model does not reproduce every detail of ISA rounding. These choices limit what I can infer about a real processor.

2 Code Changes

Brief description what you have added to the code

I reorganized the starter into separate modules so I could keep track of instruction state and pipeline activity more clearly. I explain the main changes below, followed by excerpts of the initial version and my version for each change.

Multi-issue and renaming. In **engine.py**, I added a loop that issues up to the selected width. It stops at the first instruction that cannot obtain the required resources. I process instructions in the same group one at a time, reading their source operands before assigning a new destination tag. This lets a later instruction receive the correct value from an earlier one.

Branches and memory. I added an eight-entry BTB with a one-bit predictor, along with recovery that discards younger instructions and restores register mappings. I also added forwarding from the youngest matching older store. If a load runs before an older store resolves its address and the addresses later conflict, I replay the load and the instructions after it.

Configuration, output, and tests. In **parser.py** and **model.py**, I added input parsing and configuration checks. I added the command-line interface and text/JSON output in **main.py** and **output.py**. Finally, I added automated tests, an independent sequential interpreter, and **run_experiments.py** to repeat the issue-width comparisons.

2 Code Changes - Issue and Register Renaming

2.1 Configurable issue width and ordered dispatch

I replaced the single issue call with a loop limited by the configured issue width. I still check reservation-station and ROB capacity before each instruction. If the next instruction cannot issue, I stop the loop. This lets widths 1-4 dispatch several instructions per cycle while preserving program order.

Initial version

starter_original/main.py, lines 95-102

```
'''ISSUE stage'''
if (PC.PC<len(instructions))&(PC.valid==1):
    issue(cycle, PC, instructions, ROB, size_ROB,
            rs_int_adder,
            rs_fp_adder,
            rs_fp_multi,
            ld_sd_queue, size_ld_sd_queue,
            rat_int, rat_fp)
```

My version

tomasulo/model.py, lines 20-27

```
class Config:
    issue_width: int = 1
    rob_entries: int = 128
    int_unit: UnitConfig = field(default_factory=lambda: UnitConfig(2, 1))
    fp_add: UnitConfig = field(default_factory=lambda: UnitConfig(3, 3))
    fp_mul: UnitConfig = field(default_factory=lambda: UnitConfig(2, 20))
    lsu: UnitConfig = field(default_factory=lambda: UnitConfig(3, 1))
    mem_cycles: int = 4
```

tomasulo/engine.py, lines 113-125

```
for _ in range(self.config.issue_width):
    if self.pc == len(self.program.instructions):
        break
    if not 0 <= self.pc < len(self.program.instructions):
        raise ValueError(f'Instruction PC {self.pc} is outside the program')
    instruction = self.program.instructions[self.pc]
    kind = self.kind(instruction.op)
    unit = getattr(self.config, kind)
    occupied = (len(self.lsq) if kind == 'lsu' else
                sum(e.kind == kind and e.state == 'waiting' for e in self.rob))
    if len(self.rob) >= self.config.rob_entries or occupied >= unit.rs:
        self.stats['issue_stall_cycles'] += 1
        break  # Ordered issue: never skip a blocked instruction.
```

2.2 Source operands and safe RAT updates

I kept source capture before destination renaming, which the starter already did. I changed the unsafe broadcast behavior: an older writer no longer replaces a newer RAT mapping. My RAT holds producer tags until retirement, and I remove a mapping only when the committing instruction still owns it.

Initial version

starter_original/wb.py, lines 25-31

```
reg_tag = ROB[index].dest_tag
# rat_int
if reg_tag[0] == 'R':
    rat_int[int(reg_tag[1:])] = cdb.value
# rat_fp
if reg_tag[0] == 'F':
    rat_fp[int(reg_tag[1:])] = cdb.value
```

My version

tomasulo/engine.py, lines 126-126

```
operands = [self._source(r) for r in instruction.sources]
```

tomasulo/engine.py, lines 140-142

```
# Sources must be captured before the destination's new mapping.
if instruction.dest and instruction.dest != 'R0':
    self.rat[instruction.dest] = entry.tag
```

tomasulo/engine.py, lines 374-378

```
dest = entry.instruction.dest
if dest and dest != 'R0':
    self.registers[dest] = entry.value
    if self.rat.get(dest) == entry.tag:
        del self.rat[dest]
```

2 Code Changes - Timing and Input Options

2.3 One execution advance per cycle and explicit readiness

I replaced incremented execution counters with an absolute completion cycle. The starter could call the same functional-unit update before and after adding a waiting instruction, advancing active entries twice. I also check both issue time and operand readiness, so an instruction cannot execute during its issue cycle or consume a same-cycle broadcast.

Initial version

starter_original/exe.py, lines 118-123

```
'''execution in fu and ld_sd address calculation'''
ld_sd_execution(ld_sd_exe, time_ld_sd_exe, ld_sd_queue, ROB, cycle)
# functional units
fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
fu_exe(fu_fp_adder, results_buffer, ROB, time_fu_fp_adder, cycle, PC)
fu_exe(fu_fp_multi, results_buffer, ROB, time_fu_fp_multi, cycle, PC)
```

starter_original/exe.py, lines 140-146

```
if check_valid_ins_in_rs(rs_int_adder)>=0:
    index = check_valid_ins_in_rs(rs_int_adder)
    fu_int_adder.append(fu_entry())
    add_entry_into_fu(fu_int_adder, rs_int_adder[index])
    # check if it's a waiting instruction
    if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
        fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
```

My version

tomasulo/engine.py, lines 154-156

```
def _operands_ready(self, entry, address_only=False):
    operands = entry.operands[-1:] if address_only else entry.operands
    return all(o.tag is None and o.ready_cycle < self.cycle for o in operands)
```

tomasulo/engine.py, lines 169-177

```
if (entry.kind != kind or entry.state != 'waiting' or
        entry.trace['issue'] >= self.cycle or
        not self._operands_ready(entry, kind == 'lsu')):
    continue
entry.state = 'executing'
entry.trace['ex_start'] = self.cycle
end = self.cycle + unit.ex_cycles - 1
entry.trace['ex_end'] = end
self.executing.append((end, entry.tag))
```

2.4 New command-line options and configuration validation

I added command-line options for the input file, issue width, cycle limit, and saved results. I also validate configuration values before simulation starts, including positive integers and the supported width range. These are new interfaces; the starter used hardcoded settings and only read assembly lines from code.in.

Initial version

There is no equivalent command-line argument parser or centralized configuration validator in the starter. Its input is fixed by starter_original/main.py:15, its hardware settings are hardcoded in that file, and starter_original/init.py:58-61 only reads and splits input lines.

My version

main.py, lines 14-20

```
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('input', nargs='?', default='code.in', help='assembly/configuration input')
parser.add_argument('--issue-width', type=int, choices=range(1, 5), help='override input width (1-4)')
parser.add_argument('--max-cycles', type=int, default=100000)
parser.add_argument('--json', dest='json_path', type=Path, help='write complete machine-readable results')
parser.add_argument('--output', type=Path, help='also save the text results')
parser.add_argument('--show-squashed', action='store_true', help='include wrong-path and replayed rows')
```

tomasulo/model.py, lines 32-40

```
def positive_integer(name: str, value: int) -> None:
    if type(value) is not int or value <= 0:
        raise ValueError(f"{name} must be a positive integer")

positive_integer("issue width", self.issue_width)
if self.issue_width > 4:
    raise ValueError("issue width must be between 1 and 4")
positive_integer("ROB entries", self.rob_entries)
positive_integer("memory cycles", self.mem_cycles)
```

2 Code Changes - Branches and Store Forwarding

2.5 Branch prediction and recovery after a wrong prediction

I changed branches from stopping fetch to using an eight-entry, direct-mapped branch target buffer. A missing entry follows the next instruction. When a prediction is wrong, I squash younger work, cancel its execution or memory request, rebuild the RAT from surviving ROB entries, and restart fetch at the resolved PC.

Initial version

starter_original/issue.py, lines 239-244

```
put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
# put ins into rs_int_adder
index = check_rs_space(rs_int_adder)
put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
# PC
PC.valid = 0
```

My version

tomasulo/engine.py, lines 144-151

```
if instruction.op in ('beq', 'bne'):
    prediction = self.btb[self.pc % 8]
    if prediction and prediction['pc'] == self.pc and prediction['taken']:
        next_pc = prediction['target']
    entry.predicted_pc = next_pc
    trace['predicted_pc'] = next_pc
    self.stats['branch_predictions'] += 1
self.pc = next_pc
```

tomasulo/engine.py, lines 194-209

```
def _squash(self, first_tag, new_pc, reason):
    removed = [e for e in self.rob if e.tag >= first_tag]
    if removed:
        for entry in removed:
            entry.trace.update(squashed=True, squash_cycle=self.cycle,
                               squash_reason=reason)
            self.entries.pop(entry.tag)
        self.stats['squashed'] += len(removed)
        self.rob = [e for e in self.rob if e.tag < first_tag]
        self.lsq = [t for t in self.lsq if t < first_tag]
        self.executing = [(end, t) for end, t in self.executing if t < first_tag]
        if self.memory_job and self.memory_job[1] >= first_tag:
            self.memory_job = None
    self._rebuild_rat()
    self.pc = new_pc
    self.fetch_resume = self.cycle + 2
```

2.6 Forward from the youngest older store with a matching address

I corrected the store search used by loads. The starter stops at the nearest older store even if that store has a different address. I scan backward until I find the youngest older store with the same address. If its data is not ready, the load waits instead of using stale memory.

Initial version

starter_original/mem.py, lines 19-27

```
def forward_check_from_sd(ld_sd_queue, index):
    data = []
    for i in reversed(range(index)):
        if (ld_sd_queue[i].op=='Sd'):
            if (ld_sd_queue[i].address == ld_sd_queue[index].address):
                if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
                    data.append(ld_sd_queue[i].data)
            break
    return data
```

My version

tomasulo/engine.py, lines 284-291

```
def _load_value(self, load):
    for tag in reversed(self.lsq):
        older = self.entries[tag]
        if tag < load.tag and older.instruction.op == 'sd' and older.address == load.address:
            if not self._operands_ready(older):
                return False, None, False
            return True, older.operands[0].value, True
    return True, None, False
```

2 Code Changes - Memory Replay and Checks

2.7 Detect a memory-order conflict and replay the load

I let a load proceed while an older store still has an unresolved address. When that store resolves, I check younger loads that have already started reading. If I find a conflict without an intervening matching store, I squash from that load and fetch it again so it can obtain the correct value.

Initial version

starter_original/mem.py, lines 30-40

```
def check_all_previous_Sd_no_match(ld_sd_queue, index):
    flag = True
    for i in range(index):
        # all Sd instructions
        if (ld_sd_queue[i].op=='Sd'):
            if ld_sd_queue[i].ready == 0:
                flag = False
                break
            elif (ld_sd_queue[i].address==ld_sd_queue[index].address):
                flag = False
                break
```

My version

tomasulo/engine.py, lines 211-227

```
def _check_memory_violation(self, store):
    # Loads leave the LSQ on receiving data but remain in the ROB until
    # retirement, so all speculative reads can still be checked here.
    for younger in self.rob:
        if (younger.tag > store.tag and younger.instruction.op == 'ld' and
                younger.trace['mem_start'] is not None and
                younger.address == store.address):
            # A known intervening same-address store supersedes this store.
            intervening = any(e.tag > store.tag and e.tag < younger.tag and
                              e.instruction.op == 'sd' and
                              e.address == store.address for e in self.rob)
            if intervening:
                continue
            younger.trace['replays'] += 1
            self.stats['memory_replays'] += 1
            self._squash(younger.tag, younger.instruction.pc, 'memory replay')
            return
```

How I checked these changes

I used the existing automated tests to check these behaviors. The latest recorded run passed all 44 test methods, including 72 seeded differential configurations. The simulator code is the same code used for the result tables that follow.

Code change	Checks in tests/test_simulator.py
Multi-issue dispatch	Issue-width limits, ordered stalls, and small ROB/RS capacity.
Register renaming	Same-group RAW/WAR/WAW cases and hardwired R0.
Cycle timing	Execution latency, operand readiness, pipelined FP, and one CDB.
Branch recovery	Taken branches, BTB collisions, and restoration of surviving renames.
Store forwarding	Youngest matching store and store data that is not ready.
Memory replay	Aliasing and non-aliasing stores with unresolved addresses.
Input and CLI	Invalid settings, parser errors, saved output, and cycle limits.

I show selected source ranges from the original starter and my submitted implementation. The starter functions were reorganized into Simulator methods, so the old and new filenames differ. These excerpts show parts of functions; common leading indentation is removed for readability.

3 Results

Show results of test cases under different issue width

I reran the simulator on October 15, 2026. All 44 test methods passed, including 72 seeded differential configurations. I then ran seven benchmarks at all four issue widths. All 28 runs produced the same final register and memory values as my independent sequential interpreter.

Test case	Width 1	Width 2	Width 3	Width 4	Correctness
Corrected starter loop	66	65	65	65	4 / 4 PASS
Independent mixed operations	47	48	47	47	4 / 4 PASS
Dependency / renaming chain	40	38	38	36	4 / 4 PASS
Store-to-load forwarding	44	44	44	44	4 / 4 PASS
Branch recovery	44	44	44	44	4 / 4 PASS
Memory disambiguation / replay	34	34	35	35	4 / 4 PASS
Small ROB / RS / LSQ	42	42	42	42	4 / 4 PASS

Table 1: Total simulated cycles for all seven test cases. Lower is better.

The clearest improvement was in the dependency test, which went from 40 cycles at width 1 to 36 at width 4. That is 10% fewer cycles, or a 1.11x speedup. Some other tests changed very little, and a few configurations took one extra cycle. From these results, I found that issue width helps only when there are enough ready instructions and available resources.

3.1 Test Case 1: Dependency and register renaming

I used this program to check instructions that read and overwrite the same registers. It also gives the integer and FP units work that can overlap.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	6	2	-	1
FP adder	5	3	-	1
FP multiplier	3	6	-	1
Load/store unit	3	1	4	1

ROB entries = 32
Initial values: R1=5, R2=3, F1=2, F2=3. Other registers and memory start at zero.

List of instructions

```
Add R3 R1 R2
Addi R1 R3 1
Sub R4 R1 R2
Add R3 R4 R1
Addi R5 R3 -2
Add R6 R5 R4
Add.d F3 F1 F2
Mult.d F4 F3 F2
Add.d F1 F4 F3
Sub.d F3 F1 F2
Mult.d F5 F3 F1
Add.d F6 F5 F4
Addi R3 R6 1
Add R7 R3 R1
```

3.1.1 Test Case 1 - Issue widths 1 and 2

I kept the program and hardware settings unchanged. At width 1, the issue stage admits one instruction per cycle unless it stalls; at width 2, it can admit two. I checked that dependent instructions still waited for their operands. A dash means that an instruction does not use that stage.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R3 R1 R2	1	2-3	-	4	5
Addi R1 R3 1	2	5-6	-	7	8
Sub R4 R1 R2	3	8-9	-	10	11
Add R3 R4 R1	4	11-12	-	13	14
Addi R5 R3 -2	5	14-15	-	16	17
Add R6 R5 R4	6	17-18	-	19	20
Add.d F3 F1 F2	7	8-10	-	11	21
Mult.d F4 F3 F2	8	12-17	-	18	22
Add.d F1 F4 F3	9	19-21	-	22	23
Sub.d F3 F1 F2	10	23-25	-	26	27
Mult.d F5 F3 F1	11	27-32	-	33	34
Add.d F6 F5 F4	12	34-36	-	37	38
Addi R3 R6 1	13	20-21	-	23	39
Add R7 R3 R1	14	24-25	-	27	40

Table 2: Test Case 1 at issue width 1 - 40 total cycles.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R3 R1 R2	1	2-3	-	4	5
Addi R1 R3 1	1	5-6	-	7	8
Sub R4 R1 R2	2	8-9	-	10	11
Add R3 R4 R1	2	11-12	-	13	14
Addi R5 R3 -2	3	14-15	-	16	17
Add R6 R5 R4	3	17-18	-	19	20
Add.d F3 F1 F2	4	5-7	-	8	21
Mult.d F4 F3 F2	4	9-14	-	15	22
Add.d F1 F4 F3	5	16-18	-	20	23
Sub.d F3 F1 F2	5	21-23	-	24	25
Mult.d F5 F3 F1	6	25-30	-	31	32
Add.d F6 F5 F4	6	32-34	-	35	36
Addi R3 R6 1	7	20-21	-	22	37
Add R7 R3 R1	7	23-24	-	25	38

Table 3: Test Case 1 at issue width 2 - 38 total cycles.

The wider issue stage brought the FP instructions into the machine earlier. I still saw the dependent integer instructions wait for their producers, which is the behavior I expected.

3.1.2 Test Case 1 - Issue widths 3 and 4

With widths 3 and 4, I could place more instructions in the reservation stations early in the run. Execution still followed data readiness and functional-unit availability.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R3 R1 R2	1	2-3	-	4	5
Addi R1 R3 1	1	5-6	-	7	8
Sub R4 R1 R2	1	8-9	-	10	11
Add R3 R4 R1	2	11-12	-	13	14
Addi R5 R3 -2	2	14-15	-	16	17
Add R6 R5 R4	2	17-18	-	19	20
Add.d F3 F1 F2	3	4-6	-	8	21
Mult.d F4 F3 F2	3	9-14	-	15	22
Add.d F1 F4 F3	3	16-18	-	20	23
Sub.d F3 F1 F2	4	21-23	-	24	25
Mult.d F5 F3 F1	4	25-30	-	31	32
Add.d F6 F5 F4	4	32-34	-	35	36
Addi R3 R6 1	5	20-21	-	22	37
Add R7 R3 R1	6	23-24	-	25	38

Table 4: Test Case 1 at issue width 3 - 38 total cycles.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R3 R1 R2	1	2-3	-	4	5
Addi R1 R3 1	1	5-6	-	7	8
Sub R4 R1 R2	1	8-9	-	10	11
Add R3 R4 R1	1	11-12	-	13	14
Addi R5 R3 -2	2	14-15	-	16	17
Add R6 R5 R4	2	17-18	-	19	20
Add.d F3 F1 F2	2	3-5	-	6	21
Mult.d F4 F3 F2	2	7-12	-	14	22
Add.d F1 F4 F3	3	15-17	-	18	23
Sub.d F3 F1 F2	3	19-21	-	22	24
Mult.d F5 F3 F1	3	23-28	-	29	30
Add.d F6 F5 F4	3	30-32	-	33	34
Addi R3 R6 1	4	20-21	-	23	35
Add R7 R3 R1	6	24-25	-	26	36

Table 5: Test Case 1 at issue width 4 - 36 total cycles.

All four runs ended with R1=9, R3=20, R4=6, R5=13, R6=19, and R7=29. The FP results were F1=20, F3=17, F4=15, F5=340, and F6=355. I used these final values as a correctness check as well as comparing them with the sequential interpreter.

3.2 Test Case 2: Branch prediction and recovery

I chose this program to check a loop and a taken branch that skips a store and a register update. The loop runs four times. I wanted to confirm that speculative instructions could be discarded without changing the final program result.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	12	2	-	1
FP adder	4	3	-	1
FP multiplier	2	8	-	1
Load/store unit	8	1	3	1

ROB entries = 48
Initial values: R1=0, R2=4, R3=100, F1=1, F2=2.
Mem[0]=10 and Mem[4]=20. Other registers and memory start at zero.

List of instructions

```
Addi R1 R1 1
Addi R3 R3 2
Bne R1 R2 -3
Add R4 R1 R3
Beq R1 R2 2
Sd F2 0(R0)
Addi R3 R3 999
Add.d F3 F1 F2
Sd F3 4(R0)
Addi R5 R4 1
```

Issue width	Total cycles	Committed	Mispredictions	Squashed
1	44	17	3	17
2	44	17	3	24
3	44	17	3	24
4	44	17	3	24

Table 6: Branch-recovery results at each issue width.

Every run finished in 44 cycles and committed 17 instructions. At width 1, I saw 17 instructions squashed; at widths 2-4, I saw 24. Wider issue allowed more speculative work into the machine, but it did not reduce the total time for this program. All runs ended with R1=4, R3=108, R4=112, R5=113, and F3=3. Mem[0] stayed at 10, and Mem[4] became 3.

Input correction and verification

The supplied starter loop used branch offset -12. Under the required PC+4+(offset<<2) rule, that target is outside the program. I preserved the original input and used a documented correction of -8 for the corrected-starter benchmark. That benchmark completed 25 instructions and produced the expected final values.

I checked timing rules separately from the sequential interpreter. My tests cover register dependencies, resource stalls, forwarding, branch recovery, replay, malformed input, and the command line. I have no known failing cases in the included tests. The full traces and rerun logs are in results/.

```
python3 -m unittest discover -s tests -v
python3 run_experiments.py
```